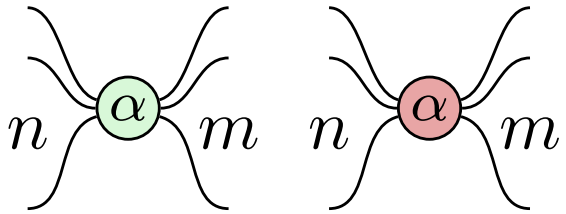
Verifying Tensor Networks in a Proof Assistant

Robert Rand
with Adrian Lehmann, Ben Caldwell, William Spencer, Bhakti Shah, and Aleks Kissinger
University of Chicago



Quantum Resource Estimation, 2026

ZX-Calculus



The **ZX-Calculus** is a graph-based alternative to the quantum circuit model.

The key benefit of using the ZX-calculus is performing diagrammatic rewrites instead of matrix computations.

By using the ZX-Calculus we can rapidly perform these computations using a small set of rewrite rules.

Why Verify ZX?

- Used in a range of useful software, from optimizers to simulators to error correcting codes.
- Guarantees the validity of existing rules.
- Allows us to extend ZX with additional rules and constructs and guarantee their correctness.

Why Rocq?

VyZX is inspired by CompCert, a verified C compiler. It is designed to verify transformations over ZX diagrams, in the same way you might verify optimizations as part of a compiler.

The VyZX project is built using Rocq for three reasons:

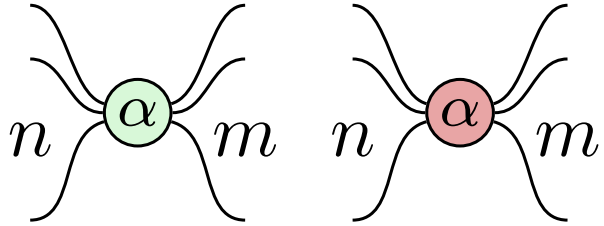
- A pre-existing library: QuantumLib;
- Interoperability with tools like sqir and voqc;
- Code Extraction.



ZX-Calculus

The ZX-Calculus is a graphical language for reasoning about quantum processes. In the ZX-Calculus, there are two colors of node and edges between nodes.

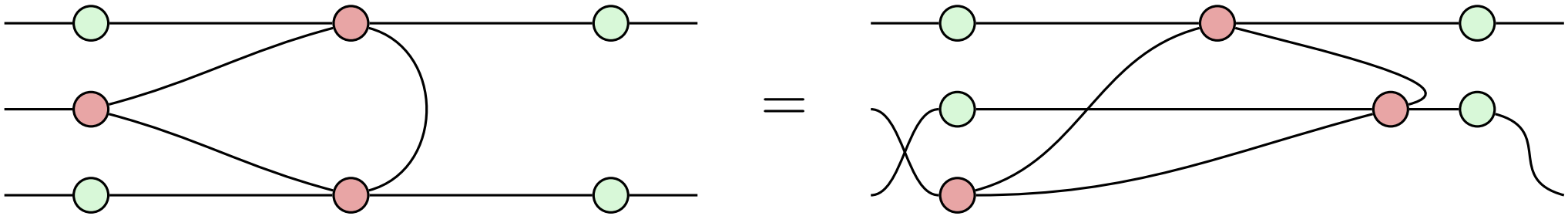
The nodes are colored either Green or Red and are called the Z or X spiders.



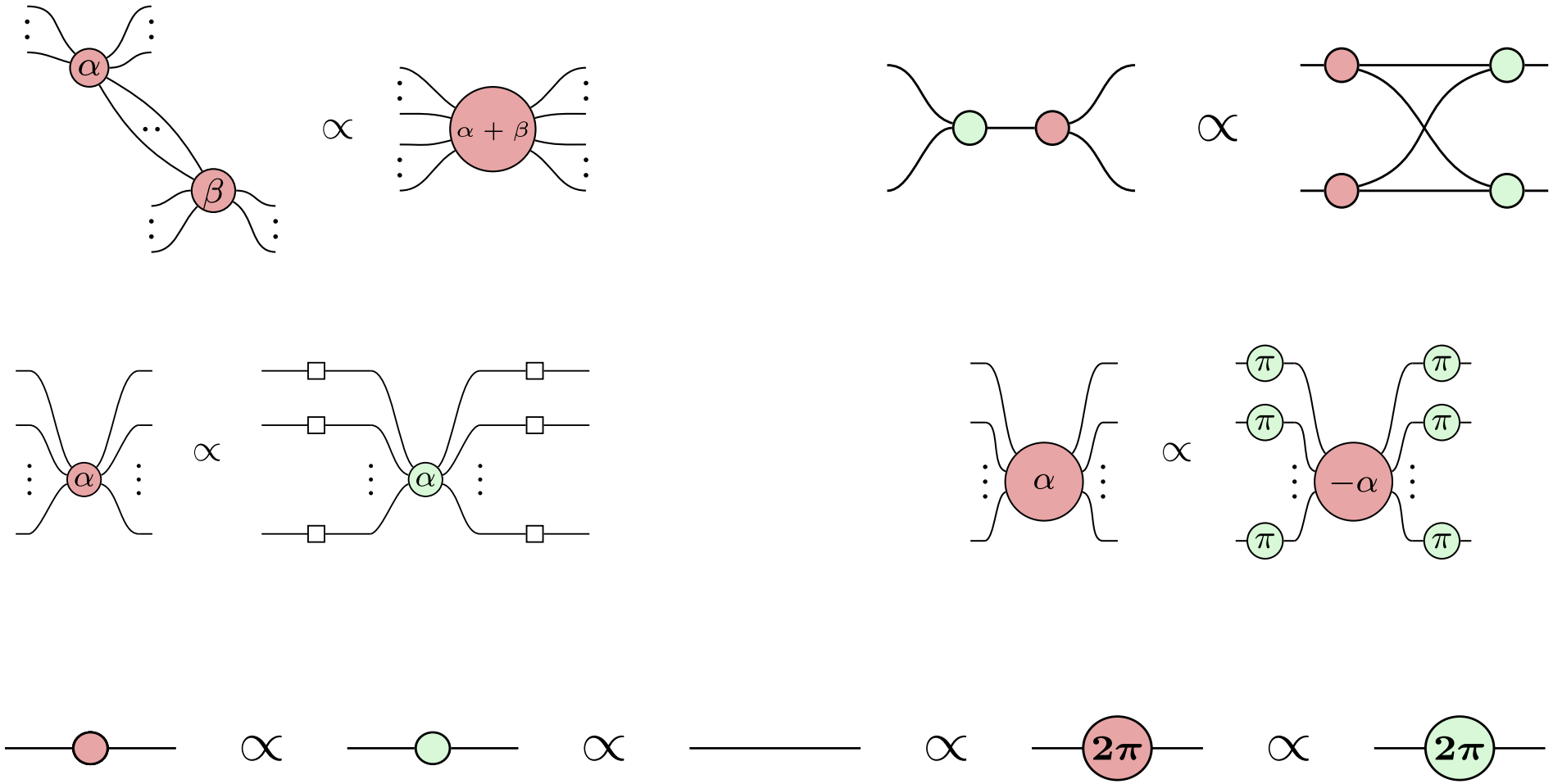
They are called Z and X spiders as for $n,m = 1$ and $\alpha = \pi$, they act as the standard Z and X gates and more generally correspond to Z and X rotations.

Only Connectivity

When working with the ZX-Calculus, the phrase **only connectivity matters** is used to describe how we can freely move spiders around, so long as their connections remain the same.



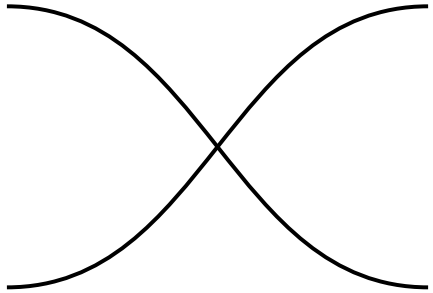
Equational Theory



Towards Semantics: ZX Diagrams By Parts

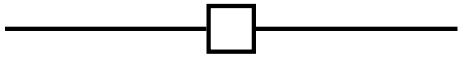


Cup and cap

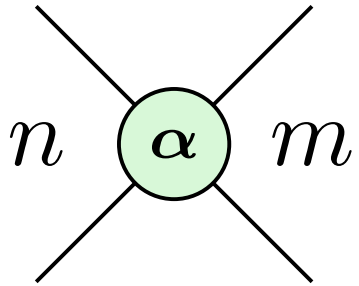


Swap

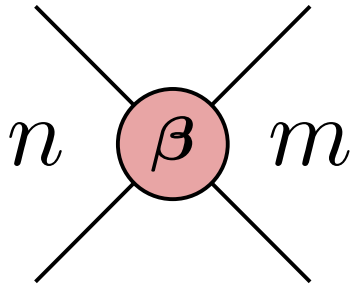
Identity wire



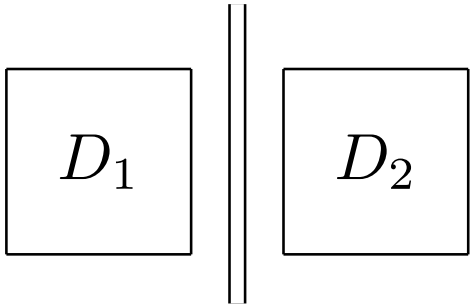
Hadamard



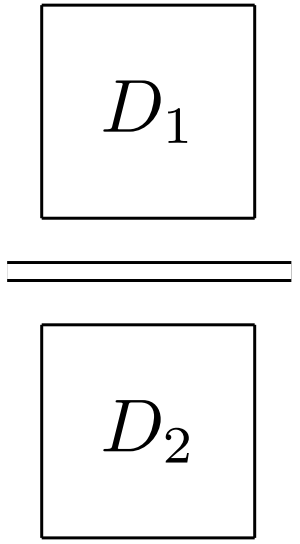
Z spider



X spider

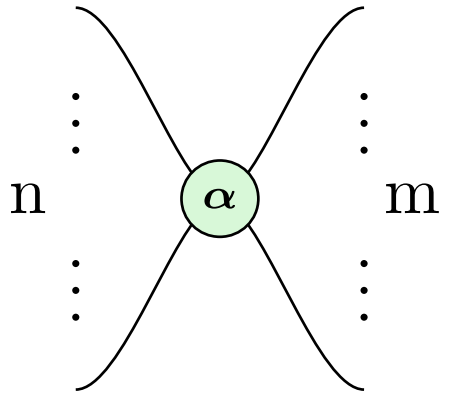


Tensor (side by side)



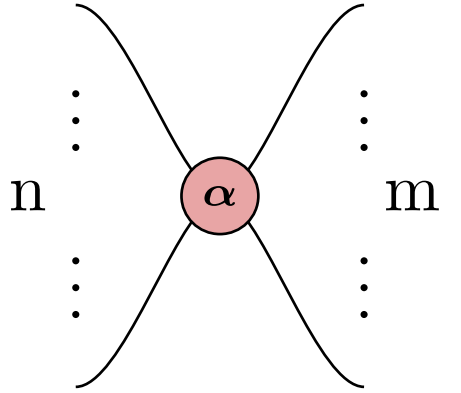
Composition (stacked)

Spider Semantics



$$= \quad \begin{matrix} n \\ |0\ldots0\rangle \end{matrix} \quad \mapsto \quad \begin{matrix} m \\ |0\ldots0\rangle \end{matrix}$$

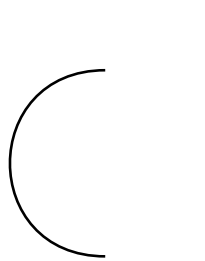
$$\begin{matrix} n \\ |1\ldots1\rangle \end{matrix} \quad \mapsto \quad \begin{matrix} m \\ e^{i\alpha}\,|1\ldots1\rangle \end{matrix}$$



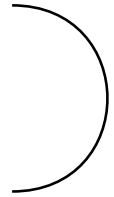
$$= \quad \begin{matrix} n \\ |+\cdots+\rangle \end{matrix} \quad \mapsto \quad \begin{matrix} m \\ |+\cdots+\rangle \end{matrix}$$

$$\begin{matrix} n \\ |-\cdots-\rangle \end{matrix} \quad \mapsto \quad \begin{matrix} m \\ e^{i\alpha}\,|-\cdots-\rangle \end{matrix}$$

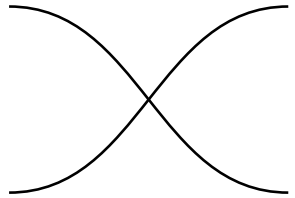
Diagram Semantics



$$\mapsto \quad \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

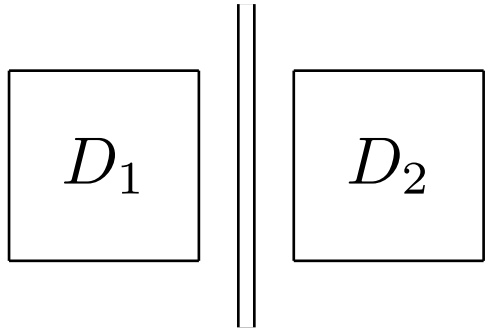


$$\mapsto \quad \begin{bmatrix} 1 & 0 & 0 & 1 \end{bmatrix}$$

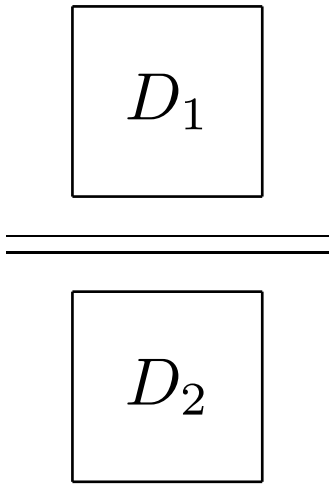


$$\mapsto \quad \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Diagram Semantics

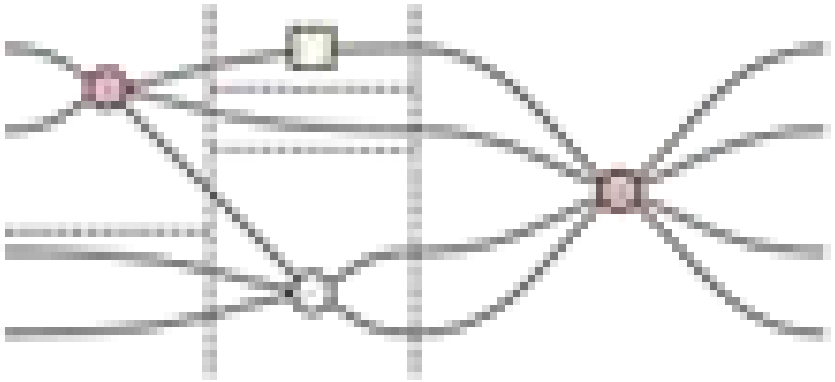


$$\mapsto \quad [\![D_2]\!] \times [\![D_1]\!]$$

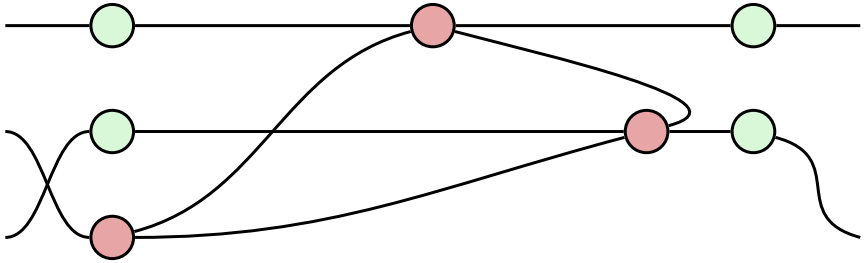


$$\mapsto \quad [\![D_1]\!] \otimes [\![D_2]\!]$$

Tension!



VS



Biting the Bullet: Inductive ZX Diagrams

To define our ZX diagrams, we take the basic compositional structure and represent it with a **dependently-typed inductive datatype**.

$$\frac{\mathrm{in}\ \mathrm{out} : \mathbb{N} \qquad \alpha : \mathbb{R}}{\mathtt{Z}\ \mathrm{in}\ \mathrm{out}\ \alpha : \mathtt{ZX}\ \mathrm{in}\ \mathrm{out}} \qquad \frac{\mathrm{in}\ \mathrm{out} : \mathbb{N} \qquad \alpha : \mathbb{R}}{\mathtt{X}\ \mathrm{in}\ \mathrm{out}\ \alpha : \mathtt{ZX}\ \mathrm{in}\ \mathrm{out}}$$

$$\frac{}{\subset : \mathtt{ZX}\ 0\ 2} \qquad \frac{}{\supset : \mathtt{ZX}\ 2\ 0} \qquad \frac{}{- : \mathtt{ZX}\ 1\ 1} \qquad \frac{}{\square : \mathtt{ZX}\ 1\ 1} \qquad \frac{}{\times : \mathtt{ZX}\ 2\ 2} \qquad \frac{}{\emptyset : \mathtt{ZX}\ 0\ 0}$$

$$\frac{\mathtt{zx}_0 : \mathtt{ZX}\ \mathrm{in}\ \mathrm{mid} \qquad \mathtt{zx}_1 : \mathtt{ZX}\ \mathrm{mid}\ \mathrm{out}}{\mathtt{zx}_0\ ;\ \mathtt{zx}_1 : \mathtt{ZX}\ \mathrm{in}\ \mathrm{out}} \qquad \frac{\mathtt{zx}_0 : \mathtt{ZX}\ \mathrm{in}_0\ \mathrm{out}_0 \qquad \mathtt{zx}_1 : \mathtt{ZX}\ \mathrm{in}_1\ \mathrm{out}_1}{\mathtt{zx}_0 * \mathtt{zx}_1 : \mathtt{ZX}\ (\mathrm{in}_0 + \mathrm{in}_1)\ (\mathrm{out}_0 + \mathrm{out}_1)}$$

Semantics (Spiders)

To verify transformations on diagrams, we give the datatype their desired semantics. Our semantics relies on QuantumLib, a mathematical library specialized for quantum computing.

$$[\![\mathrm{Z}\ \mathrm{n}\ \mathrm{m}\ \alpha]\!] = \begin{bmatrix} 1 & \cdots & 0 \\ \vdots & \ddots & 0 \\ 0 & 0 & e^{i\alpha} \end{bmatrix}$$

$$[\![\mathrm{X}\ \mathrm{n}\ \mathrm{m}\ \alpha]\!] = H^{\otimes m} \times \begin{bmatrix} 1 & \cdots & 0 \\ \vdots & \ddots & 0 \\ 0 & 0 & e^{i\alpha} \end{bmatrix} \times H^{\otimes n}$$

Semantics (everything else)

$$[\![\text{Cup}]\!] = [1 \quad 0 \quad 0 \quad 1]^{\mathrm{T}}$$

$$[\![\text{Cap}]\!] = [1 \quad 0 \quad 0 \quad 1]$$

$$[\![\text{Swap}]\!] = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$[\![\text{Empty}]\!] = [1]$$

$$[\![\text{Compose } \text{zx}_1 \ \text{zx}_2]\!] = [\![\text{zx}_2]\!] \times [\![\text{zx}_1]\!]$$

$$[\![\text{Stack } \text{zx}_1 \ \text{zx}_2]\!] = [\![\text{zx}_1]\!] \otimes [\![\text{zx}_2]\!]$$

Proportionality

To say two diagrams are equivalent (up to a scalar we *usually* don't care about), we define a new relation on diagrams using our above semantics.

Our relation is called **`proportional`** and uses the symbol $\propto$. Our relation is defined as

$$\exists c \neq 0 : [\![zx_1]\!] = c * [\![zx_2]\!] \Longrightarrow zx_1 \propto zx_2$$

This will allow us to use Rocq's **rewriting capabilities** in our proofs about diagrams.

Proportionality++

We also provide additional notions of precise semantic equivalence ($\propto_=$) and equivalence up to a specific scalar ($\propto_c$), which are defined as you'd expect.

These refine proportionality

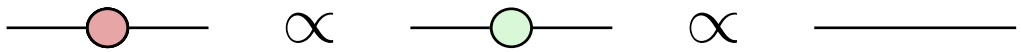
$$zx_1 \propto_= zx_2 \implies \exists c, zx_1 \propto_c zx_2 \implies zx_1 \propto zx_2$$

which means that we can use $\propto_=$ to rewrite in $\propto_c$ and $\propto_=$, and $\propto_c$ to rewrite in $\propto_=$.

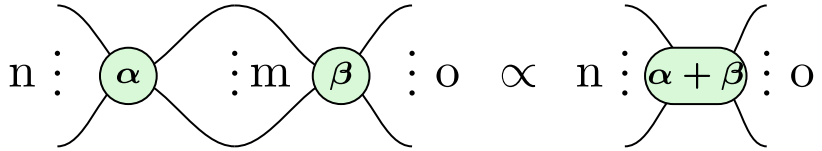
We can also construct a **scalar gadget** to convert from $\propto$ and $\propto_c$ to $\propto_=$!
(Though converting from the first one is computationally a bad idea.)

Three Proof Strategies

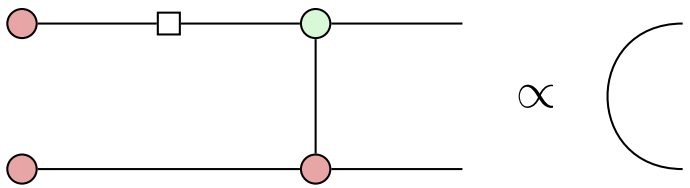
1. Proof through **semantics**



2. **Inductive** proof

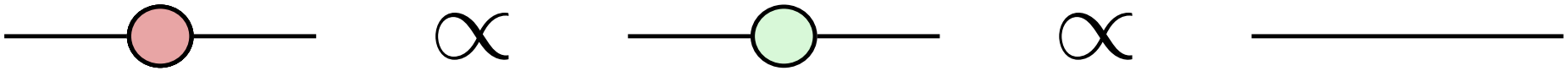


3. **Diagrammatic** proof



Proof Through Semantics

1. Proof through **semantics**

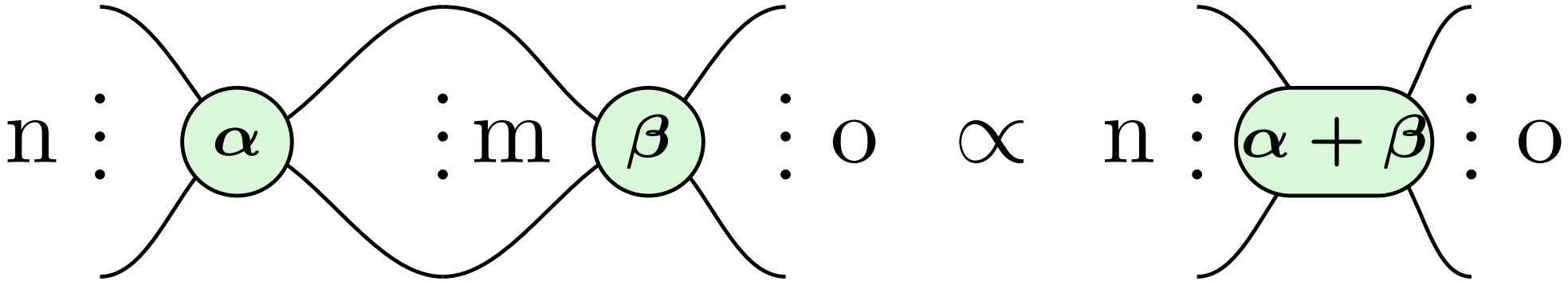


$$\begin{bmatrix} 1 & 0 \\ 0 & e^0 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I$$

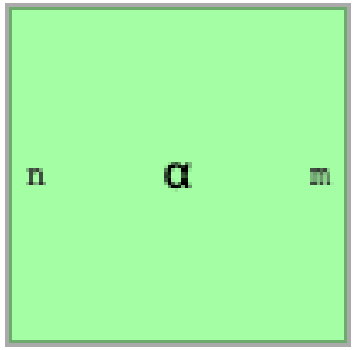
$$H \times \begin{bmatrix} 1 & 0 \\ 0 & e^0 \end{bmatrix} \times H = H \times I \times H = H \times H = I$$

Inductive Proof

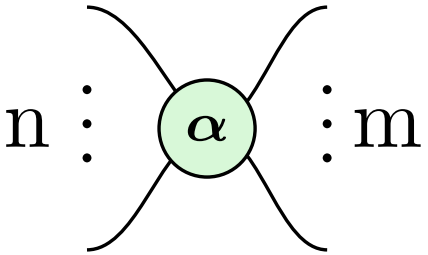
2. **Inductive** proof



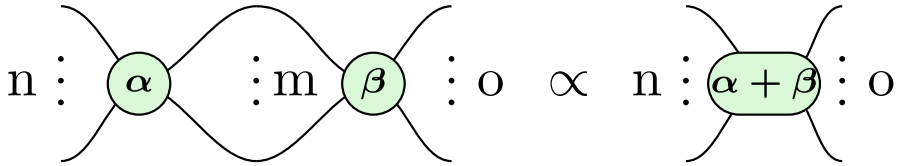
ZXViz it



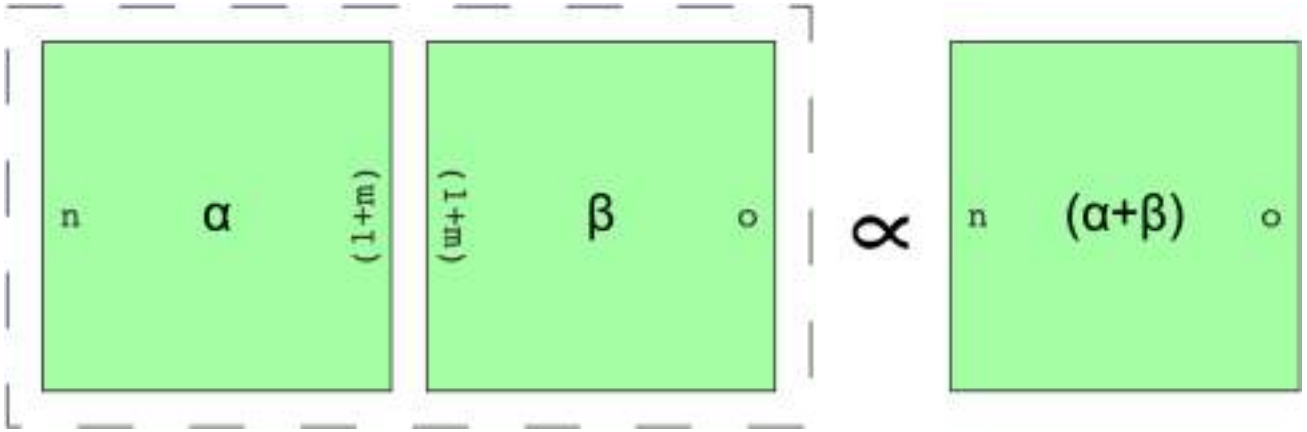
≙



2. Inductive proof: Absolute Fusion

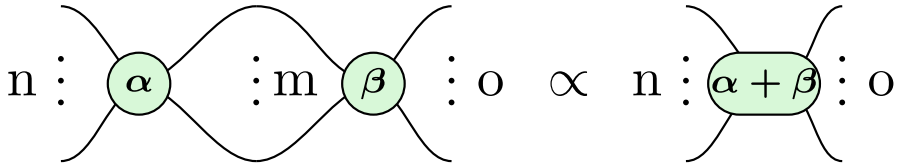


The lemma:

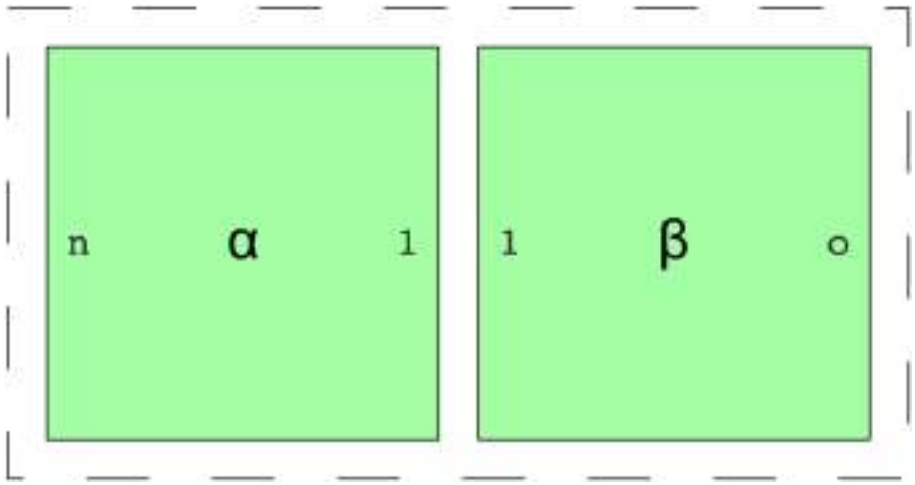


Let’s do induction over *m*

2. Inductive proof: Absolute Fusion

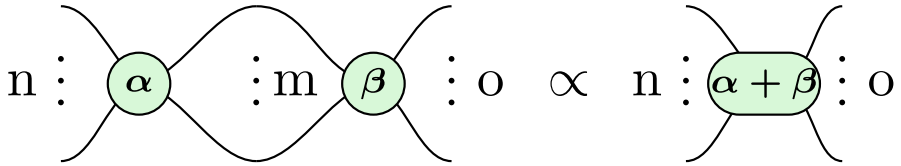


Base case:

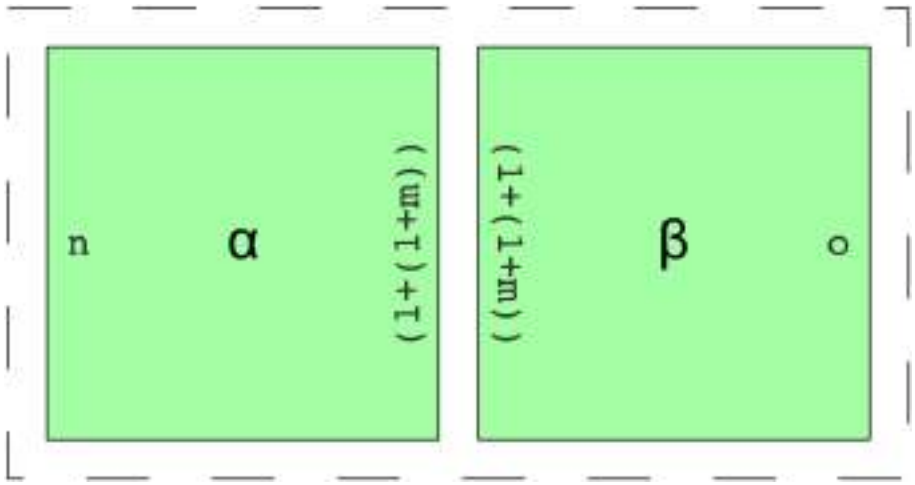


Solve with matrix semantics!

2. Inductive proof: Absolute Fusion

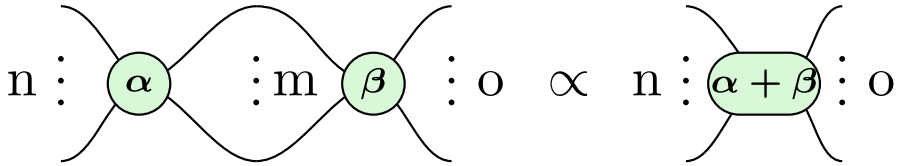


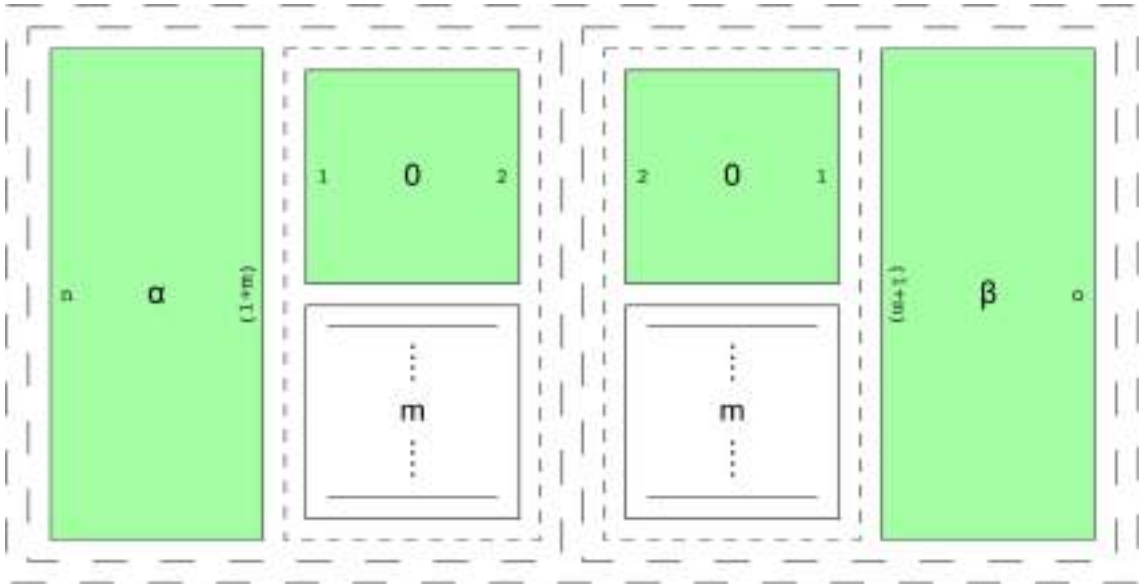
Inductive step:



Let's split out nodes to reduce size

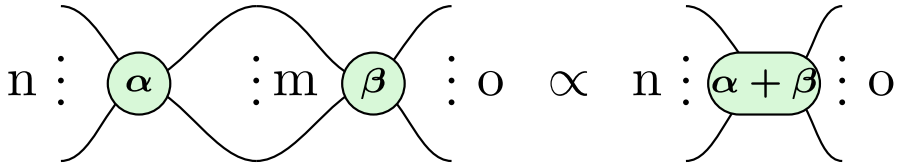
2. Inductive proof: Absolute Fusion



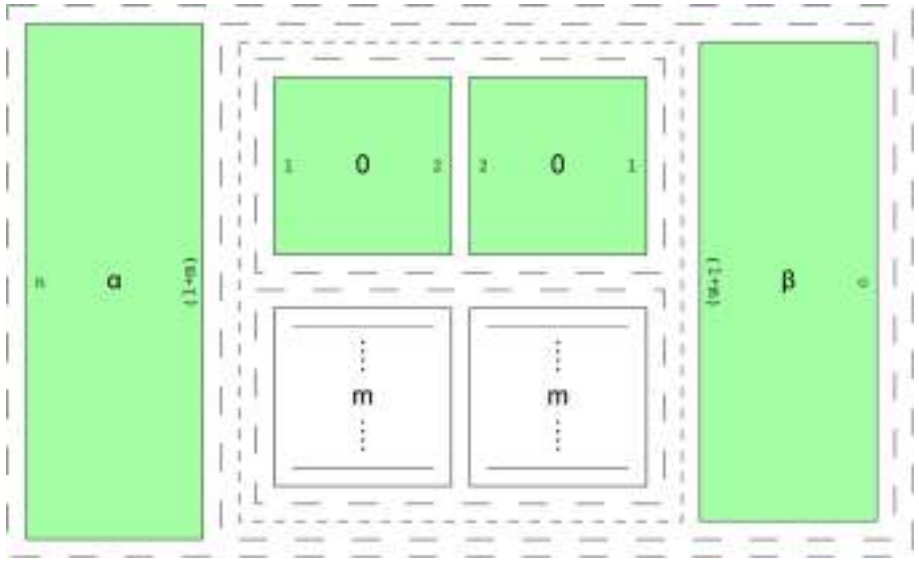


Idea: Fuse the small spiders. Problem: Association

2. Inductive proof: Absolute Fusion

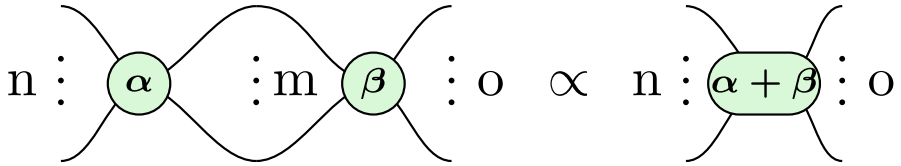


Reassociated diagram

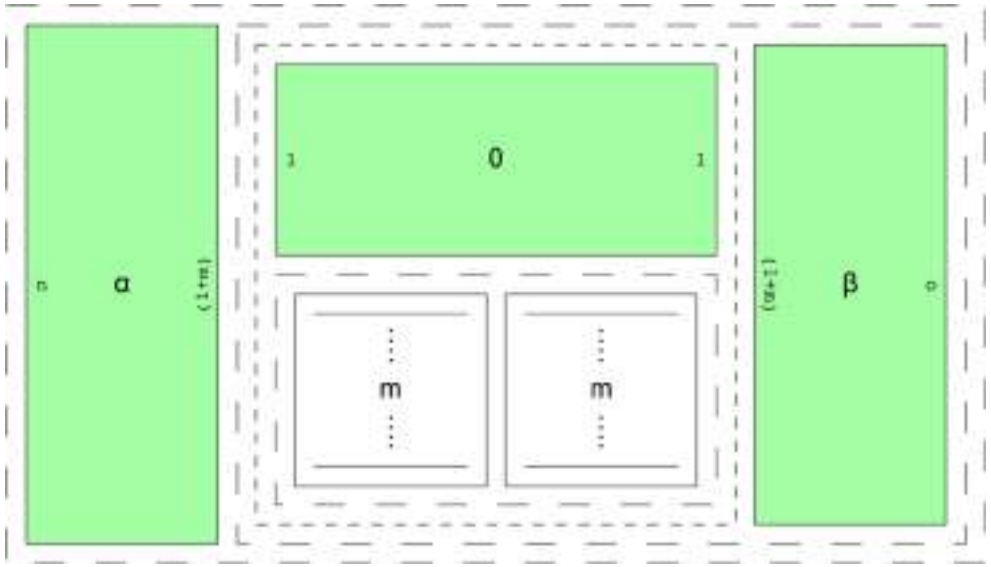


Fusing time

2. Inductive proof: Absolute Fusion

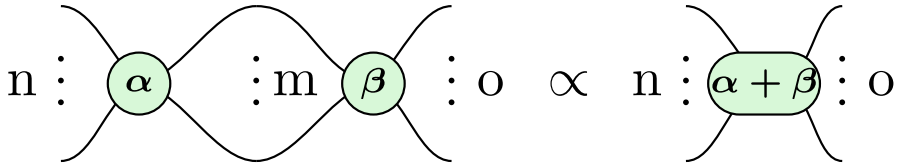


Upper spiders are fused

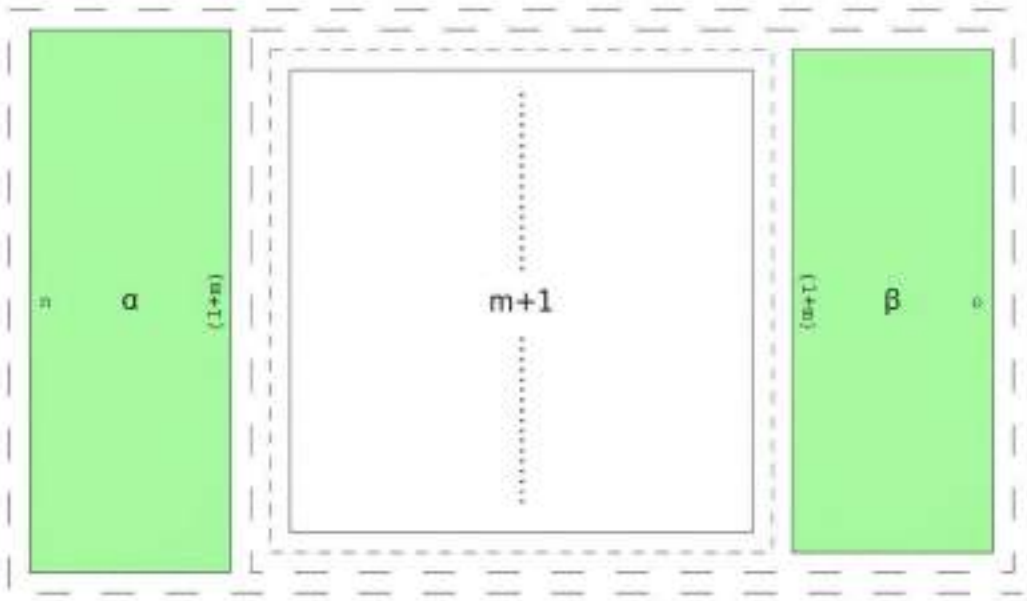


Apply identity rule, combine identities

2. Inductive proof: Absolute Fusion



Upper spiders are fused



Apply identity rule, remove wires

2. Inductive proof: Absolute Fusion

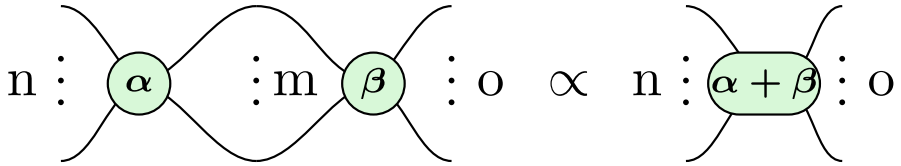
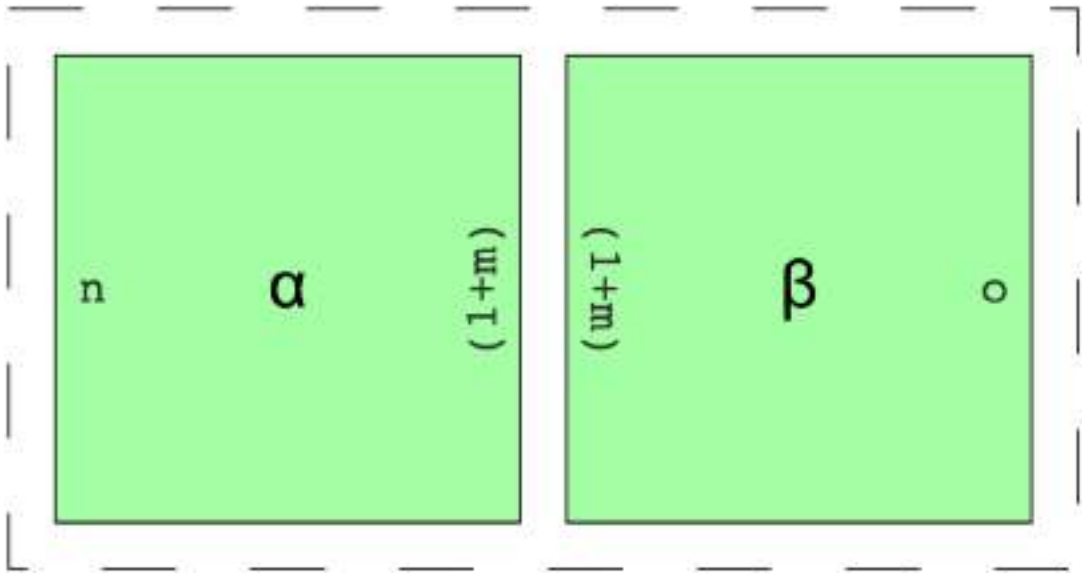
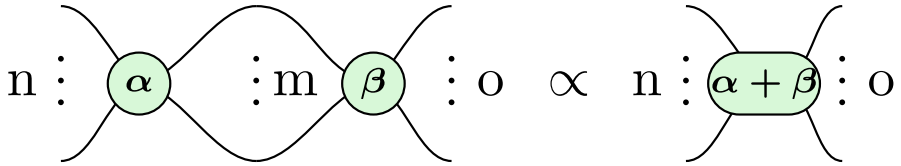


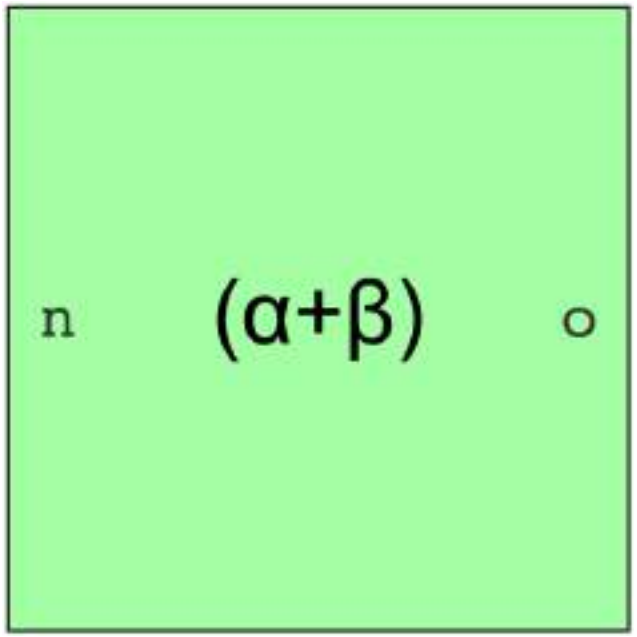
Diagram now corresponds to the Inductive Hypothesis



Apply IH.

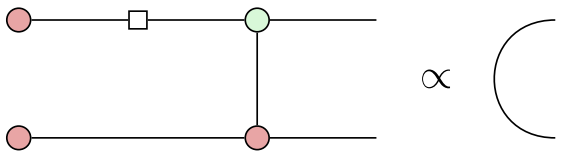
2. Inductive proof: Absolute Fusion



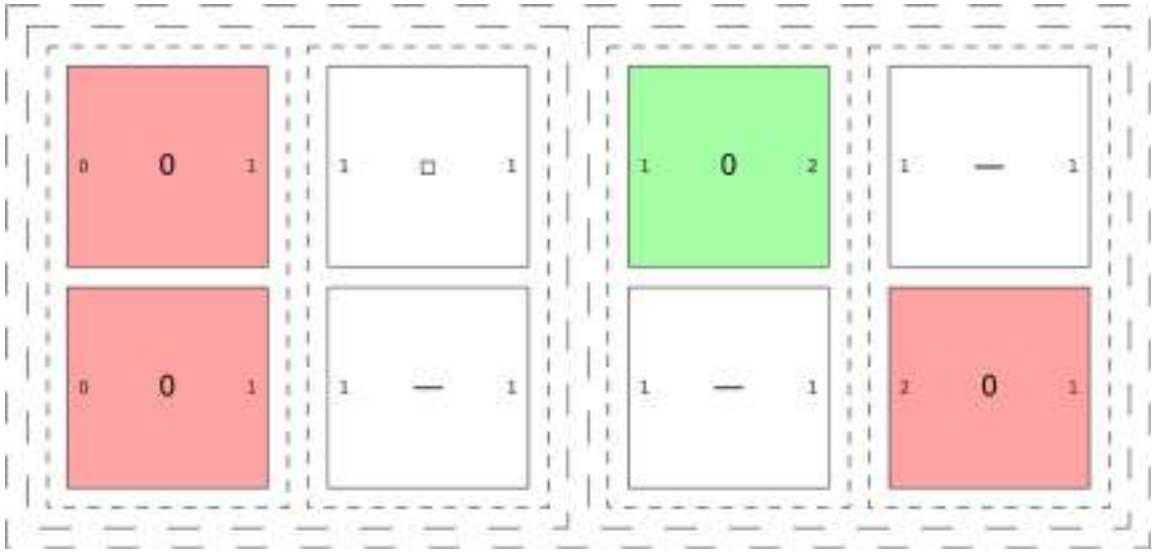


Qed.

3. Diagrammatic proof: Bell pair preparation

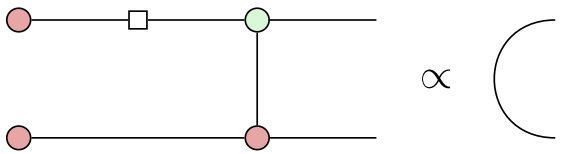


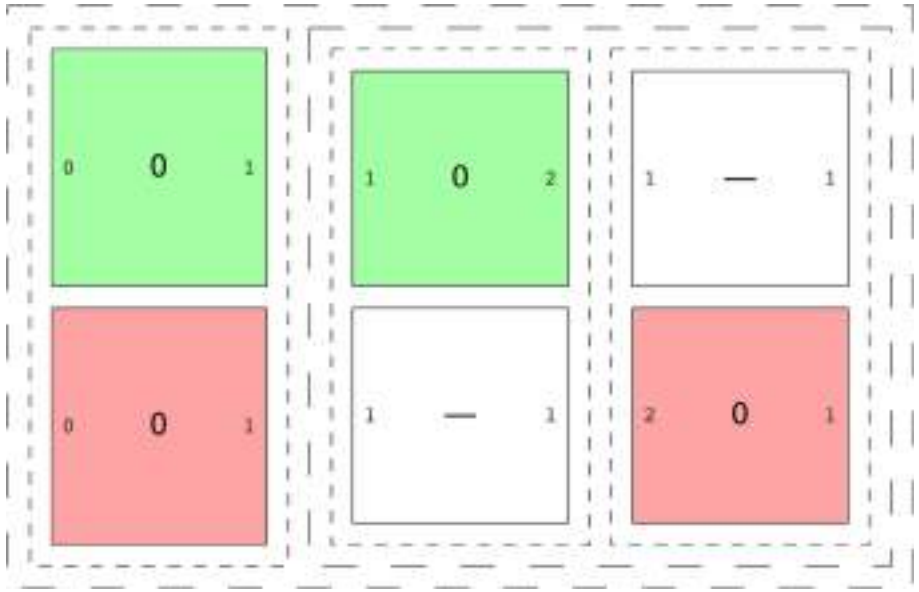
The lemma:



Let's first swap colors on the left

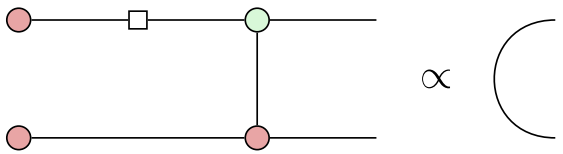
3. Diagrammatic proof: Bell pair preparation

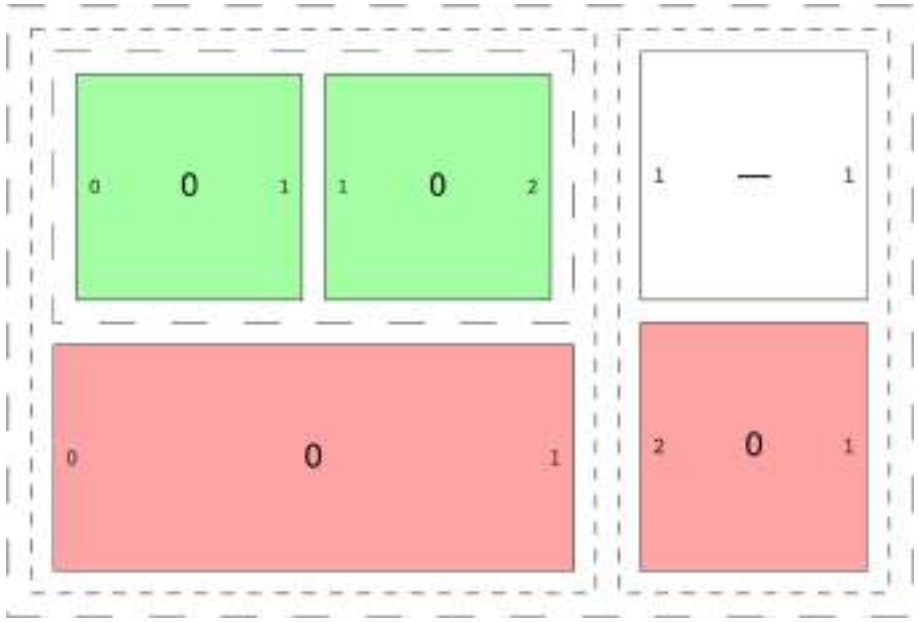




Let's reassociate!

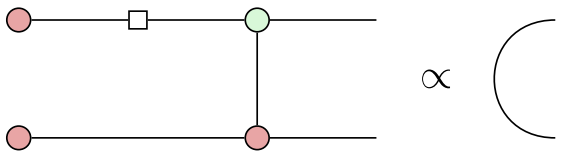
3. Diagrammatic proof: Bell pair preparation

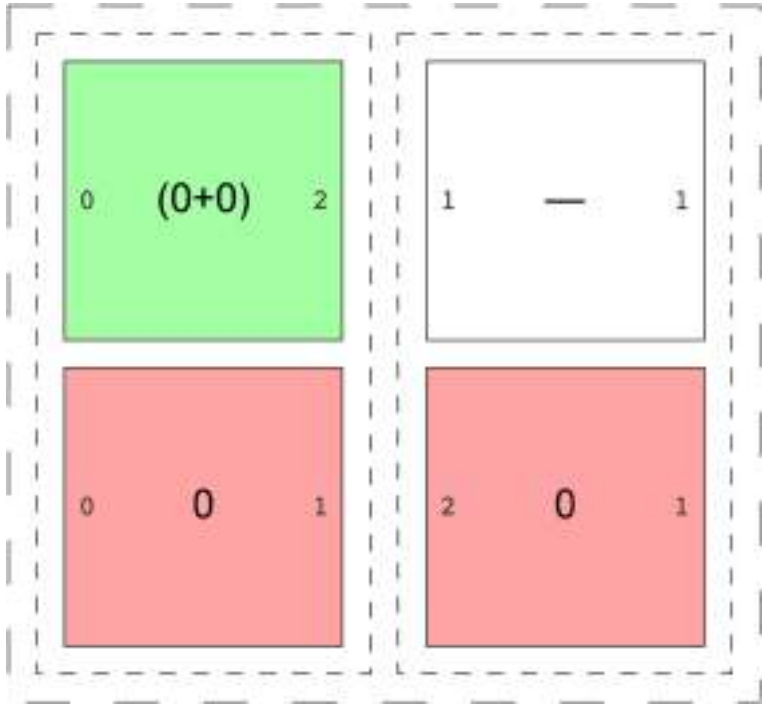




Let's fuse green nodes

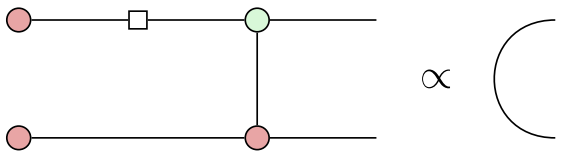
3. Diagrammatic proof: Bell pair preparation

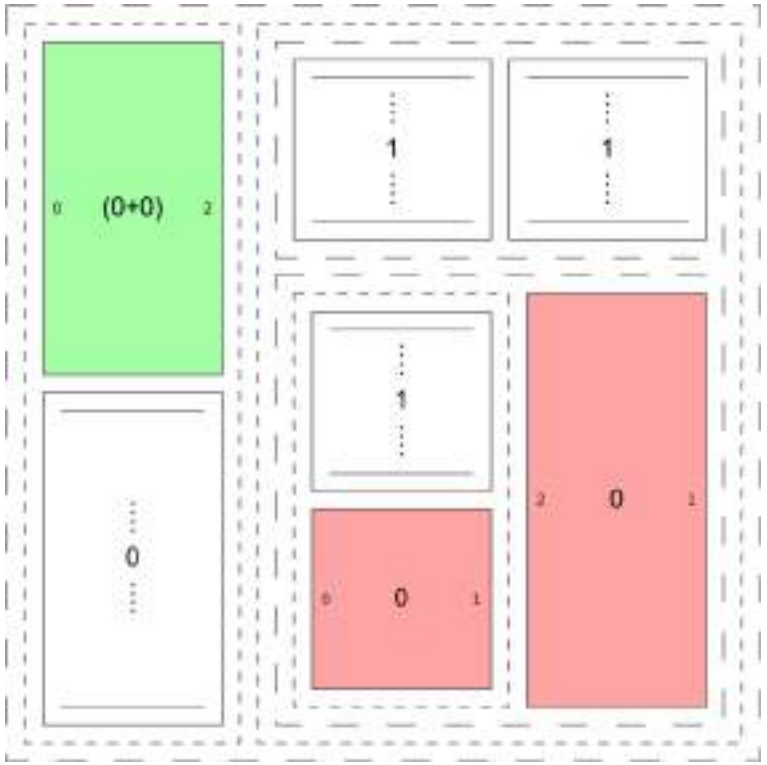




Separate the red node to fuse

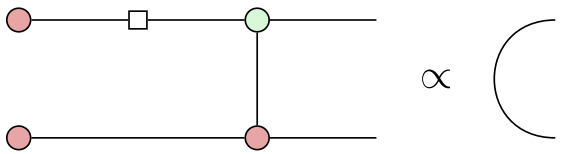
3. Diagrammatic proof: Bell pair preparation

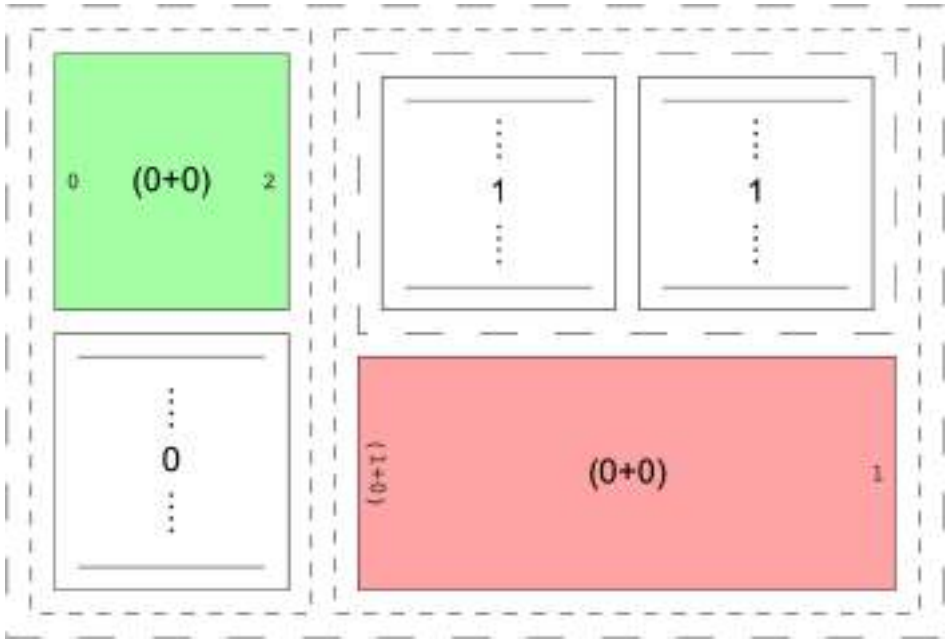




Fuse the red spiders

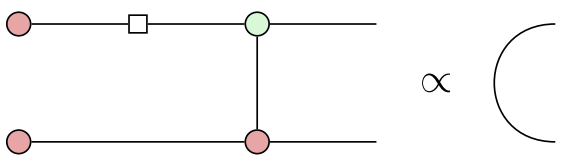
3. Diagrammatic proof: Bell pair preparation



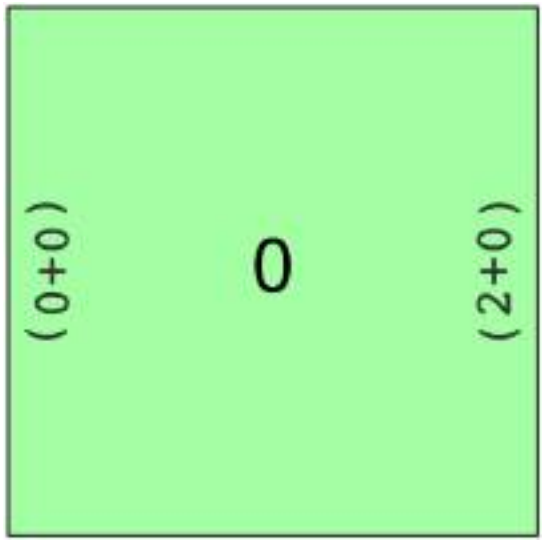


Apply identity rule, remove wires

3. Diagrammatic proof: Bell pair preparation



$\propto$

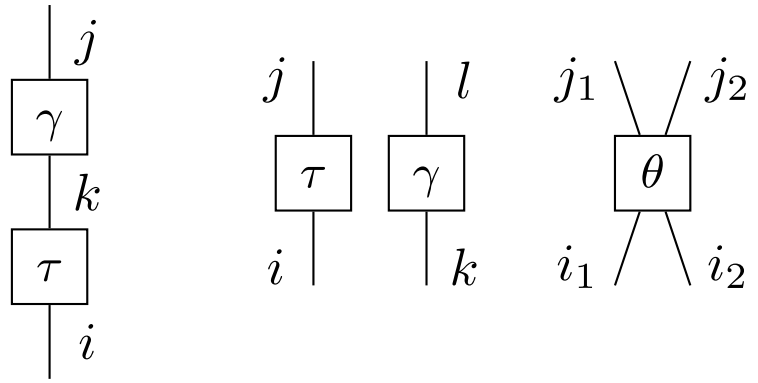


Qed.

Limitations

- Block-like structures, not graphs
- Reassociating is a barrier to proof
- Not very much in the spirit of ZX-calculus

Solution: Tensors!



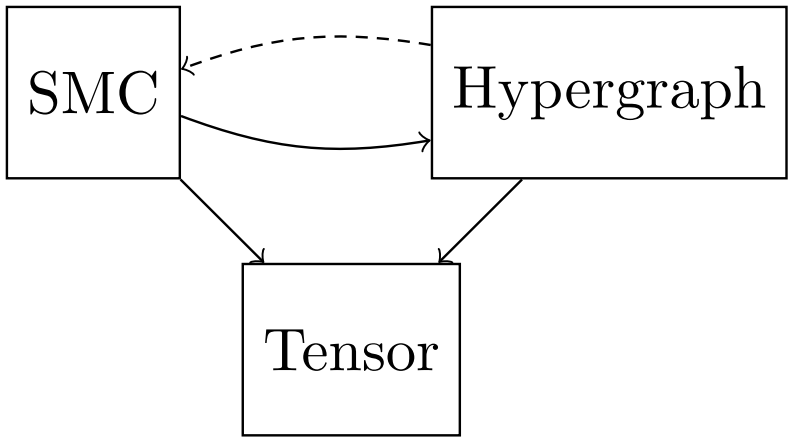
- Tensors over a field $\mathbb{F}$ act as an abstracted form of linear algebra.
- Parallel composition ≡ Tensor Product ($\otimes$).
- Sequential composition ≡ Tensor Contraction (summing over shared indices).
- Connection between processes is *algebraically encoded*.

$$\sum_k \tau_i^k * \delta_k^j = \tau_i^j$$

TensorRocq

TensorRocq is built over the connections between SMC terms and Hypergraphs.

- Shifts from strict compositionality to graph connectivity
- Automatically resolves equations derivable from SMC axioms
- Write terms compositionally, but rewrite in a hidden graph layer



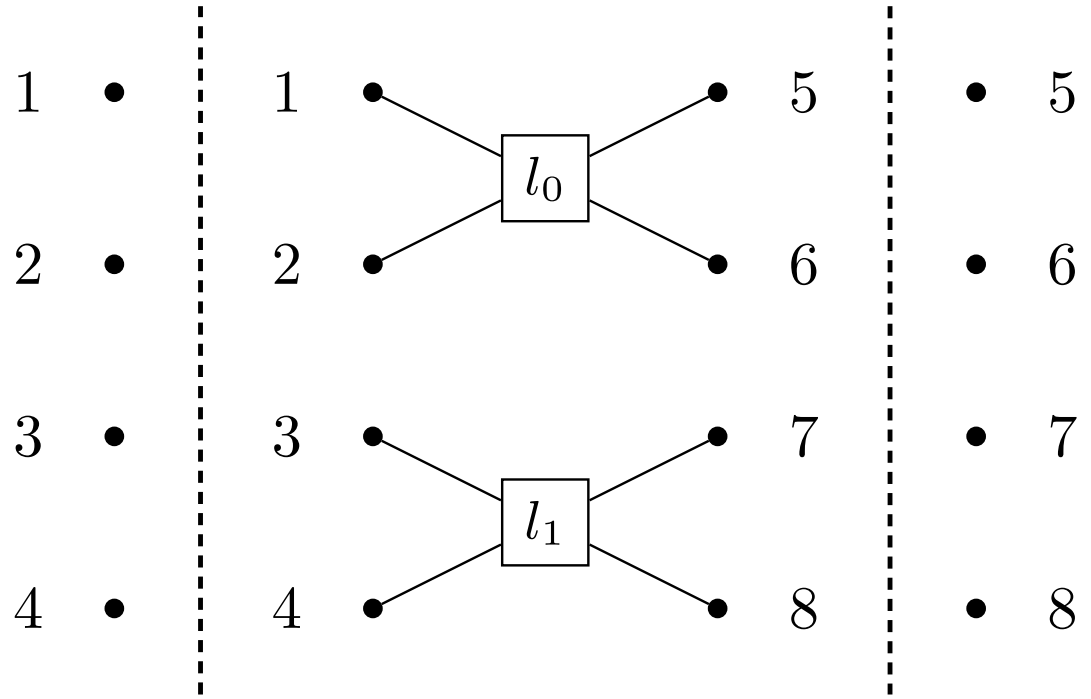
The Shallow Embedding of Tensors

```
Definition Tensor (R : Type) (n m : nat) (A : Type) :=
  vec A n -> vec A m -> R.
```

- Shallow embedding of tensors means we cannot inspect its structure.
- Defining tensors as functions gives generality to their application, perfect for denotational semantics.

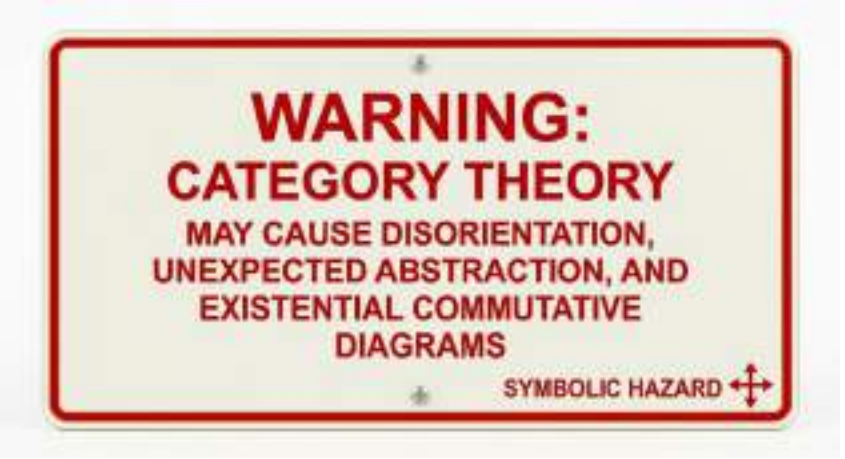
Syntactic Layer: Hypergraphs

- Edges represent processes; vertices represent connectivity.
- Interfaces fix strict ordering on inputs and outputs (V_L , V_R).
- Isomorphic hypergraphs yield identical tensor semantics.



Syntactic Layer: Autonomous PROPs

- PROPS: Enhanced with structural units (η_n) and counits (ε_n)
- APROPs: Symmetric monoidal categories where objects are natural numbers
- Serves as the primary user-facing inductive language in TensorRocq

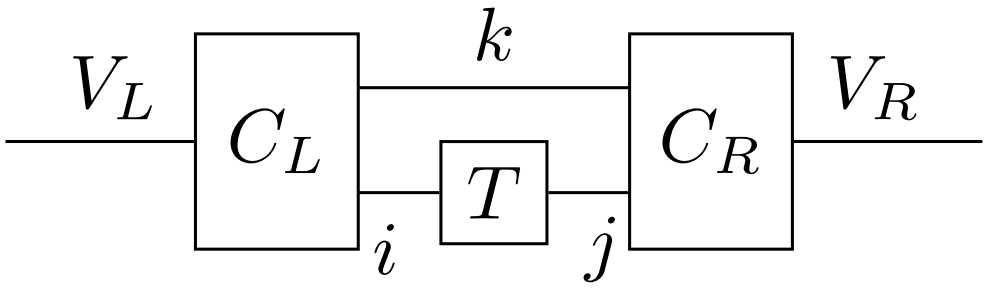


Implementing Autonomous PROPs

```
Inductive APROP {T : Type} : nat -> nat -> Type :=
  | Aid n : APROP n n
  | Aswap n m : APROP (n + m) (m + n)
  | Acup n : APROP 0 (n + n)
  | Acap n : APROP (n + n) 0
  | Acompose {n m o} (ap1 : APROP n m) (ap2 : APROP m o) : APROP n o
  | Astack {n1 m1 n2 m2} (ap1 : APROP n1 m1)
   (ap2 : APROP n2 m2) : APROP (n1 + n2) (m1 + m2)
  | Agen (t : T) n m : APROP n m.
    (* Generators of the category, encoded using T *)
```

Proof by Reflection

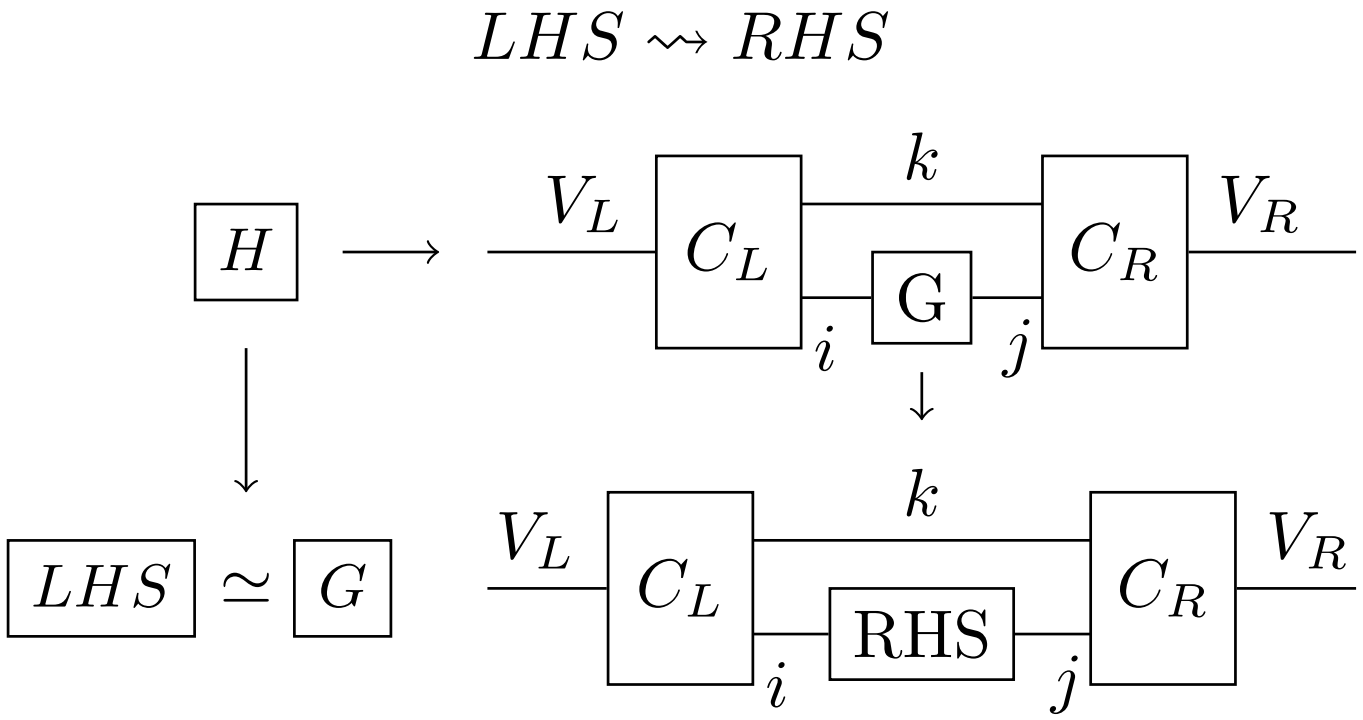
- Graph isomorphism checking reduces to pure computation via Rocq's reduction engine.
- Type labels drastically shrink the search space.
- Solves in under a second for small real-world non-synthetic verification goals.



$$T \rightsquigarrow R$$

The Rewriting Lifecycle

- Given a rule *LHS* ⇝ *RHS* and a hypergraph *H*, we find a subhypergraph *G* isomorphic to *LHS*.
- Construct the double pushout of *G* from *H*.
- Using the context of that pushout, replace *G* with *RHS*.



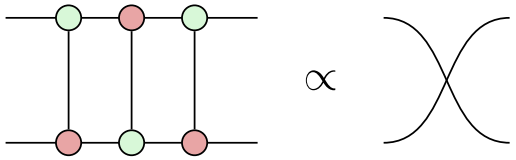
Working with Real World Projects

- Extensible pipeline avoids forcing pre-existing libraries to rewrite their datatypes.
- Quoting: Automatically ingests domain-specific terms into an internal APROP.
- Denoting: Translates the rewritten APROP smoothly back to the target language.

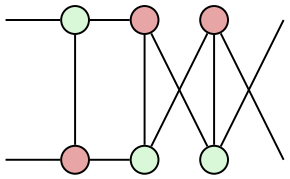
```
Instance zx_quote_n_wire n :
    Quote (n_wire n) (Aid n).
Instance zx_denote_n_wire n :
    Denote (n_wire n) (Aid n).
```

Case Study: 3-CNOT-SWAP

```
Lemma _3_cnot_swap_is_swap :
    _3_CNOT_SWAP_ ∝ SWAP.
```

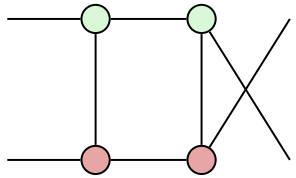


```
  rewrite cnot_is_swap_notc at 2.
  rewrite notc_is_notc_r.
```

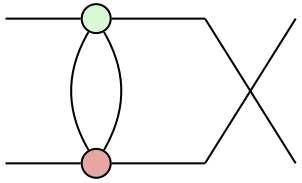


Case Study: 3-CNOT-SWAP

```
    zxrw (to_gadget bi_algebra_rule_X_over_Z).
```

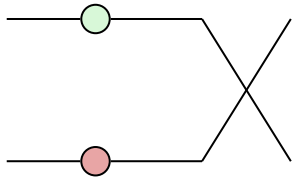


```
zxrw (@dominated_Z_spider_fusion_top_left 2 0 1 1 0 0).
rewrite Rplus_0_l.
zxrw (@dominated_X_spider_fusion_bot_right 2 0 1 1 0 0).
rewrite Rplus_0_l.
```

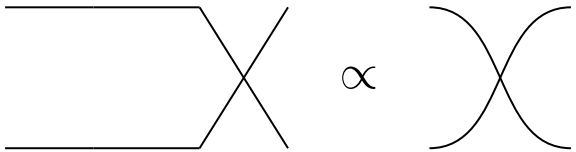


Case Study: 3-CNOT-SWAP

```
zxrw (to_gadget hopf_rule_Z_X_vert 1 1 1 1 0 0 eq_refl).
```



```
  zxrw (symmetry (zx_of_const_mult (/ 2) (/ √ 2))).
  rewrite Cinv_mult_distr.
  rewrite Z_is_wire, X_0_is_wire.
  zxcat.
```



Future Work

- Interactive visualizer for TensorRocq terms
- Porting the hypergraph rewrite engine to the Lean Prover
- Applications:
  - Graphical Linear Algebra
  - Knot and Braid Theory
  - **Quantum Software**
- Onboard users (use TensorRocq!)

